

## EXPERIMENT 7

### Endorsement Policy Configuration in Hyperledger Fabric

#### Aim

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

#### Theory

An endorsement policy defines which organizations must approve (endorse) a transaction before it can be committed. OR('Org1MSP.peer','Org2MSP.peer') allows either organization to endorse, while AND('Org1MSP.peer','Org2MSP.peer') requires both organizations.

#### Step 1: Navigate to the Test Network

```
cd ~/fabric-dev/fabric-samples/test-network
```

---

```
dhmarshna@DHARSHU:~$ cd ~/fabric-dev/fabric-samples/test-network  
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

#### Step 2: Start the Fabric Network

```
./network.sh createChannel -ca
```

---

```
++ configtxlator proto_encode --input /home/dhmarshna/fabric-dev/fabric-samples/test-network/channel-artifacts/config_update_in_envelope.json --type common.Envelope --output /home/dhmarshna/fabric-dev/fabric-samples/test-network/channel-artifacts/Org2MSPanchors.tx  
2026-09-08 16:44:45.136 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized  
2026-09-08 16:44:45.147 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update  
Anchor peer set for org 'Org2MSP' on channel 'mychannel'  
Channel 'mychannel' joined  
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

#### Step 3: Deploy Chaincode with OR Endorsement Policy

```
./network.sh deployCC \  
-ccl javascript \  
-ccn basic \  
-ccp ../asset-transfer-basic/chaincode-javascript \  
-ccep "OR('Org1MSP.peer','Org2MSP.peer')"
```

---

```
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.  
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic  
+ res=0  
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':  
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]  
Query chaincode definition successful on peer0.org2 on channel 'mychannel'  
Chaincode initialization is not required  
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

#### Step 4: Verify Chaincode Commitment

```
peer lifecycle chaincode querycommitted -C mychannel
```

---

```
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer lifecycle chaincode querycommitted \
-C mychannel
Committed chaincode definitions on channel 'mychannel':
Name: basic, Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

## Step 5: Initialize the Ledger

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function": "InitLedger", "Args": []}'
```

---

```
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function": "InitLedger", "Args": []}'
2026-09-08 16:55:00.455 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

## Step 6: Query All Assets

```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["GetAllAssets"]}'
```

---

```
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red",
"ID":"asset2","Owner":"Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":
:10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedVal
ue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asse
t6","Owner":"Michel","Size":15,"docType":"asset"}]
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

## Step 7: Stop the Network

```
./network.sh down
```

---

```
dharsna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 10/10
✓Container ca_org2 Removed 10.8s
✓Container ca_org1 Removed 10.6s
✓Container orderer.example.com Removed 0.6s
✓Container peer0.org2.example.com Removed 1.2s
✓Container peer0.org1.example.com Removed 1.3s
✓Container ca_orderer Removed 10.5s
✓Volume compose_orderer.example.com Removed 0.1s
✓Volume compose_peer0.org1.example.com Removed 0.0s
✓Volume compose_peer0.org2.example.com Removed 0.0s
✓Network fabric_test Removed 0.3s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629-c68f8ecc2a0ff0cc14976
15f5e33a8bfa908e0fd3555f864cf1398b52d341cb:latest
Deleted: sha256:291f288c01235caac85f05f1b6ed330857d0c0a7b003b9e864744f3ff1afe683
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f5be664c87b24c035bee15cbcc7f59b629-d3cf9b1f4b9747cc8433a
c0e74bfaeade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:afecb6618eda62284db2be82c3c85f20a0fc19f78573f303308b16dc3404384a
dharsna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

## Step 8: Restart the Network

*`./network.sh up createChannel -ca`*

---

```
2026-09-08 17:03:15.508 UTC 0001 INFO [channelCmd] InitCmdFactory -> Endorser and orderer connections initialized
2026-09-08 17:03:15.519 UTC 0002 INFO [channelCmd] update -> Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
Channel 'mychannel' joined
dharsna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

## Step 9: Deploy Chaincode with AND Endorsement Policy

*`./network.sh deployCC \`  
`-ccl javascript \`  
`-ccn basic \`  
`-ccp ../asset-transfer-basic/chaincode-javascript \`  
`-ccep "AND('Org1MSP.peer','Org2MSP.peer')"`*

---

```
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vsc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
dharsna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ |
```

## Step 10: Submit a Transaction

*`peer chaincode invoke \`  
`-o localhost:7050 \`  
`--ordererTLShostnameOverride orderer.example.com \`  
`--tls \`  
`--cafile $ORDERER_CA \`  
`-C mychannel \`  
`-n basic \`  
`-c`*

```
'{"function": "CreateAsset", "Args": ["asset25", "Blue", "10", "Arun", "1200"]}'
```

---

```
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile $ORDERER_CA \
-C mychannel \
-n basic \
-c '{"function": "CreateAsset", "Args": ["asset25", "Blue", "10", "Arun", "1200"]}'
2026-09-08 17:09:12.481 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset25","Color":"Blue","Size":10,"Owner":"Arun","AppraisedValue":1200}
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

## Step 11: Verify the Asset

```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["ReadAsset", "asset25"]}'
```

---

```
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["ReadAsset", "asset25"]}'
Error: endorsement failure during query. response: status:500 message:"The asset asset25 does not exist"
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

## Step 12: Stop the Network

```
./network.sh down
```

---

```
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 10/10
✓ Container peer0.org1.example.com Removed 1.2s
✓ Container orderer.example.com Removed 0.5s
✓ Container ca_org2 Removed 10.6s
✓ Container peer0.org2.example.com Removed 1.4s
✓ Container ca_orderer Removed 10.8s
✓ Container ca_org1 Removed 10.9s
✓ Volume compose_peer0.org2.example.com Removed 0.1s
✓ Volume compose_peer0.org1.example.com Removed 0.1s
✓ Volume compose_orderer.example.com Removed 0.1s
✓ Network fabric_test Removed 0.4s
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629--c68f8ecc2a0ff0cc14976
15f5e33a8bfa908e0fd3555f864cfa1398b52d341cb:latest
Deleted: sha256:2da84b4091f91d9eb3366acdb2977ead82b77127bf4d5167f56427a4c1964684
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5f6e664c87b24c035bee15cbcc7f59b629--d3cf9b1f4b9747cc8433a
c0e74bfaeade1cf0b97f5c1163835f604aa885d60dc:latest
Deleted: sha256:d363ec4ff536ff404e69515fad603b4f02031956a3c054bcc56afc0017f5a28
dhmarshna@DHARSHU:~/fabric-dev/fabric-samples/test-network$
```

## Expected Result

The endorsement policies are configured successfully. The OR policy permits endorsement by either organization, while the AND policy requires endorsements from both Org1 and Org2 before the transaction is accepted.

### **Result**

The endorsement policies were successfully configured and transactions were validated according to the specified endorsement rules, demonstrating trust, integrity, and secure transaction processing.